Course Project: One-Layer Neural Network Implementation

Objective

Implement a one-layer (single hidden layer) neural network from scratch for both **classification** and **regression** tasks, following an interface similar to scikit-learn's `MLPClassifier` and `MLPRegressor`.

Background

You will build two classes: - `SimpleSLPClassifier`: For binary and multi-class classification - `SimpleSLPRegressor`: For regression tasks

Both should use: - Forward propagation through one hidden layer - Backpropagation for gradient computation - Gradient descent optimization - Common activation functions (ReLU, sigmoid, tanh)

Task Description

Implement both neural network classes by completing the provided scaffold files: `activations.py`, `slp_base.py`, `slp_classifier.py`, and `slp_regressor.py`.

Implementation Requirements

1. Basic requirements (50%)

- Implement forward propagation with one hidden layer
- Sigmoid/logistic activation function
- Implement appropriate output layers:
  - Sigmoid for binary classification
  - Linear/identity for regression
- Implement backpropagation to compute gradients
- Update weights and biases using stochastic gradient descent
- Follow scikit-learn's interface conventions
- Track loss over iterations
- Tests run and pass

2. Advanced requirements (30%)

- Use mini-batch or full-batch gradient descent
- Use vectorized NumPy operations (no explicit loops over samples or weights)
- Implement early stopping
- Code quality (documenting comments, clear variable names, parameter validation, etc.)

3. Report (20%)

- Include a demonstration section that:
  - Tests regressor on the diabetes dataset from sklearn (`load_diabetes`)
  - Tests classifier on the breast cancer dataset from sklearn (`load_breast_cancer`) for binary classification
  - Another demonstration on different datasets of your choice (e.g. from kaggle)
  - Compares your implementation's performance with sklearn's MLPClassifier/MLPRegressor
  - Plots training loss curves
  - Reports accuracy/R^2 scores

4. Bonus (up to extra 10%, provided all basic requirements are met)

- Support multi-class classification
  - Softmax output

  - Tests classifier on a multiclass dataset from sklearn (`load_iris` or `load_digits`)
- Advanced optimization techniques, like adaptive gradient descent or momentum
- Extend to support arbitrary depth (list of layer sizes)
- Additional tests

Deliverable

A fork of this repository with the completed implementation and report. You will have to defend your work in a 1:1 meeting.

Mathematical Background

Forward Propagation (One Hidden Layer)

Hidden layer:

```
z1 = X @ W1 + b1
a1 = activation(z1)
```

Output layer:

```
z2 = a1 @ W2 + b2
y_pred = output_activation(z2)
```

Backpropagation

Compute gradients using chain rule: 1. Output layer error 2. Hidden layer error
3. Weight and bias gradients

Loss Functions

- **Classification**: Cross-entropy loss
- **Regression**: Mean squared error (MSE)

Tips

- Normalize/standardize input features and continuous targets for better convergence
- Start with a simple test case to debug (e.g., XOR problem, simple linear regression)
- Monitor for numerical stability (e.g., overflow in exp() for softmax or underflow in log() for cross-entropy)

Resources

- Tensorflow Playground
- scikit-learn MLPClassifier documentation
- scikit-learn MLPRegressor documentation
- scikit-learn datasets documentation

Due date: 2026-11-24